```
        options.AddPolicy("AllowShoppingApp", policy =>    ❷
            policy.WithOrigins("http://shopping.com")      ❸
                .AllowAnyMethod());                        ❹
    });
    // other service configuration
}
```

❶ The AddCors method exposes an Action<CorsOptions> overload.

❷ Every policy has a unique name.

❸ The WithOrigins method specifies which origins are allowed. Note that the URL has no trailing /.

❹ Allows all HTTP verbs to call the API

WARNING When listing origins in `WithOrigins()`, ensure that they don't have a trailing `"/"`; otherwise, the origin will never match, and your cross-origin requests will fail.

Once you've defined a CORS policy, you can apply it to your application. In the following listing, you apply the `"AllowShoppingApp"` policy to the whole application using `CorsMiddleware` by calling `UseCors()`.

Listing 29.3 Adding the CORS middleware and configuring a default CORS policy

```
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddCors(options => {
    options.AddPolicy("AllowShoppingApp", policy =>
         policy.WithOrigins("http://shopping.com")
             .AllowAnyMethod());
});

var app = builder.Build();
app.UseRouting();
app.UseCors("AllowShoppingApp");     ❶
app.UseAuthentication();
app.UseAuthorization();

app.MapGet("/api/products", () => new string[] {});

app.Run();
```